

**EX.NO.:** 05

**DATE:** 28.01.2025

## LATENT SPACE REPRESENTATION

### AIM

To Do the vector representations of each epoch in the latent space evolve during training on various datasets and visualize the t-SNE and PCA plotting.

### DATASET DESCRIPTION

Dataset Name: Fashion MNIST

Description:

- Dataset containing grayscale images of 10 different types of clothing items
- **Size:** Each image is 28x28 pixels in grayscale.
- **Total Samples:** The dataset contains 70,000 images, split into 60,000 training samples and 10,000 test samples.
- **Type:** Grayscale images, 28x28 pixels.

### TECHNIQUES USED

1. Autoencoder (Neural Network):
  - Architecture: The autoencoder consists of two main components: an encoder and a decoder.
    - Encoder: Compresses the input image into a lower-dimensional latent space.
    - Decoder: Attempts to reconstruct the original image from the latent space representation.
  - Activation Functions:
    - ReLU: Used for encoding and decoding layers to introduce non-linearity.
    - Sigmoid: Used in the output layer to normalize the reconstructed image between 0 and 1.
  - Loss Function: Mean Squared Error (MSE) is used as the loss function since it measures the difference between the original and the reconstructed images.
2. Dimensionality Reduction:
  - PCA (Principal Component Analysis): PCA is used to reduce the high-dimensional latent space of the encoder into a 2D space. It is a linear technique that finds the principal components that explain the maximum variance in the data.
    - Helps to visualize the latent space and understand how the model has learned to represent the data in fewer dimensions.
  - t-SNE (t-Distributed Stochastic Neighbor Embedding): t-SNE is a non-linear dimensionality reduction technique that maps high-dimensional data into 2D or 3D space while preserving the local structure (i.e., similar data points remain close in the 2D visualization).
    - Unlike PCA, t-SNE is better at capturing non-linear relationships in the data and is commonly used for visualizing clusters.
3. Training and Visualization:
  - Epochs: The autoencoder is trained for 10 epochs, with the latent space visualized after each epoch. This helps track the evolution of the latent space and see how the model improves its representations over time.
  - Batch Size: A batch size of 256 is used during training for efficiency.

### CODE AND OUTPUT

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from tensorflow.keras.datasets import fashion_mnist

# Load Fashion MNIST dataset
```

```

(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# Preprocess the data
X_train = X_train.astype('float32') / 255.0 # Normalize to [0, 1]
X_test = X_test.astype('float32') / 255.0 # Normalize to [0, 1]

# Flatten images
X_train_flat = X_train.reshape(X_train.shape[0], -1) # (60000, 784)
X_test_flat = X_test.reshape(X_test.shape[0], -1) # (10000, 784)

# Build the Autoencoder model
input_dim = X_train_flat.shape[1]
input_layer = layers.Input(shape=(input_dim,))
encoder = layers.Dense(128, activation='relu')(input_layer)
latent_space = layers.Dense(64, activation='relu', name="latent_space")(encoder)
decoder = layers.Dense(128, activation='relu')(latent_space)
output_layer = layers.Dense(input_dim, activation='sigmoid')(decoder)
autoencoder = models.Model(input_layer, output_layer)

# Compile the autoencoder
autoencoder.compile(optimizer='adam', loss='mse')

# Extract the encoder model for latent space visualization
encoder_model = models.Model(input_layer, latent_space)

# Function to visualize latent space with PCA or t-SNE
def visualize_latent_space(data, title, method='pca'):
    if method == 'pca':
        reducer = PCA(n_components=2)
    elif method == 'tsne':
        reducer = TSNE(n_components=2, random_state=42)
    else:
        raise ValueError("Method must be 'pca' or 'tsne'.")

    reduced_data = reducer.fit_transform(data)
    plt.figure(figsize=(8, 6))
    scatter = plt.scatter(reduced_data[:, 0], reduced_data[:, 1], c=y_test, s=10,
alpha=0.7)
    plt.colorbar(scatter)
    plt.title(title)
    plt.xlabel("Component 1")
    plt.ylabel("Component 2")
    plt.show()

# Train the autoencoder and visualize latent space at each epoch
epochs = 10
batch_size = 256

for epoch in range(epochs):

```

```

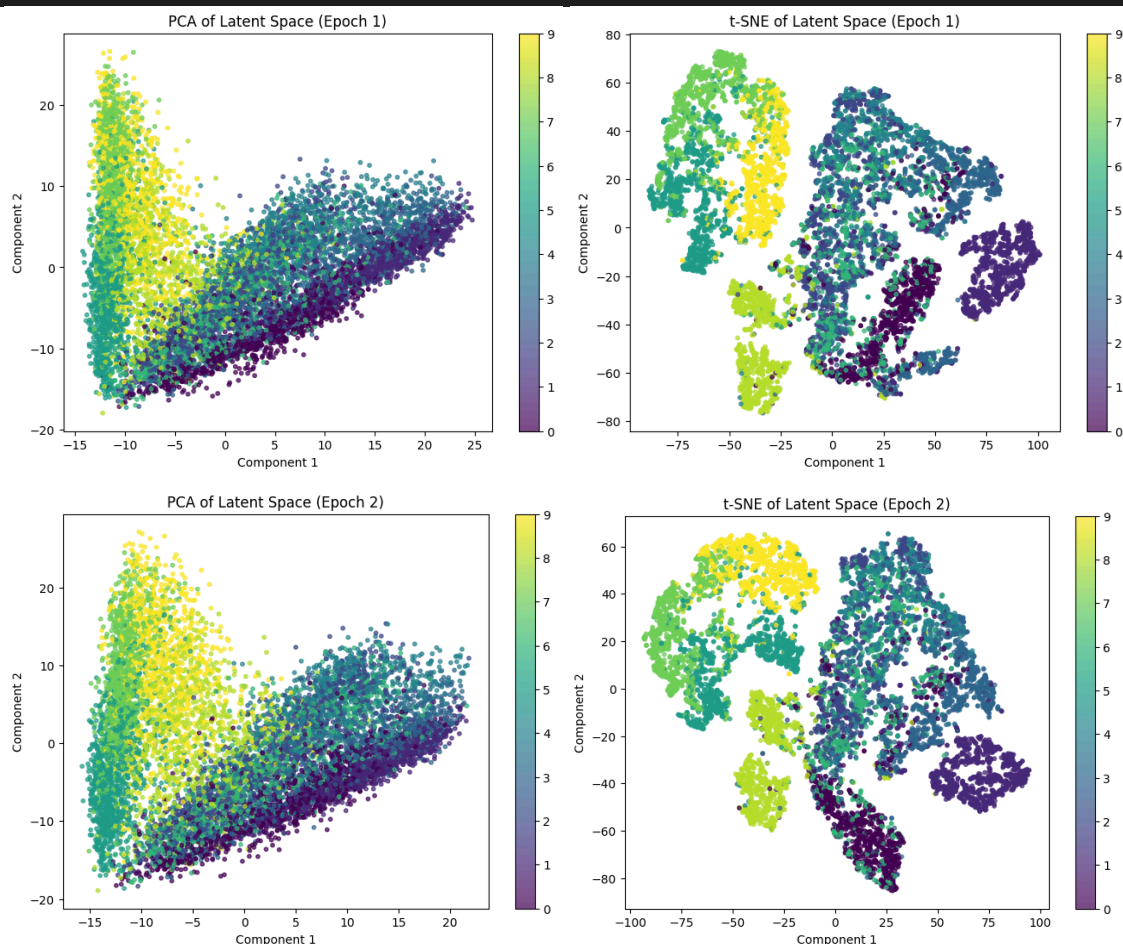
print(f"\nEpoch {epoch + 1}/{epochs}")
autoencoder.fit(X_train_flat, X_train_flat, epochs=1, batch_size=batch_size,
verbose=1, validation_data=(X_test_flat, X_test_flat))

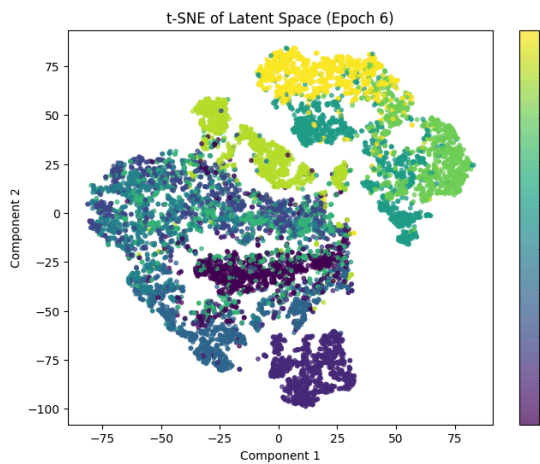
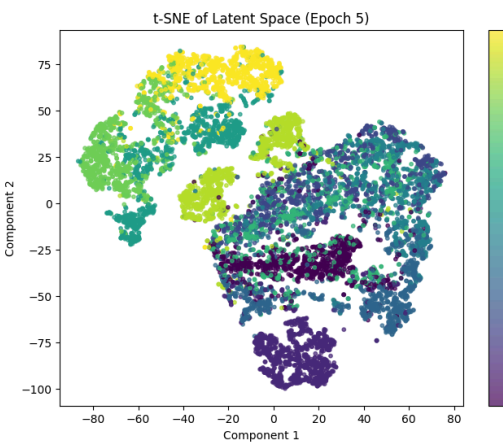
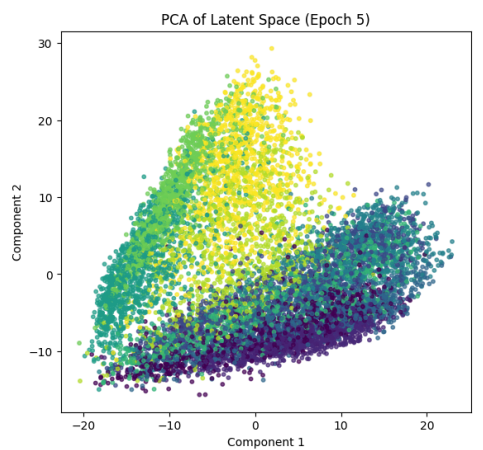
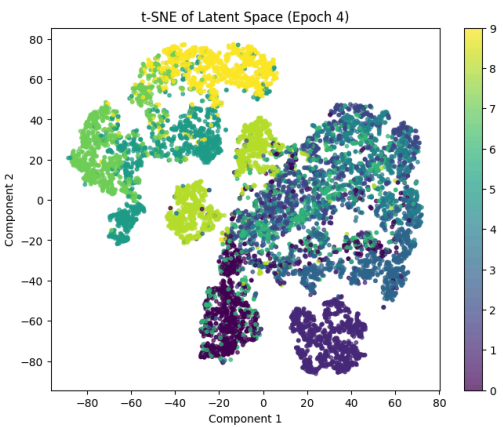
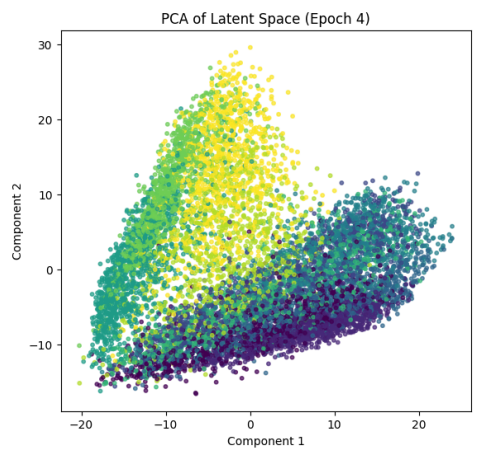
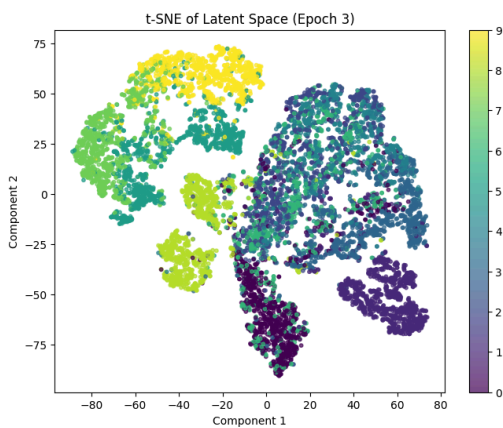
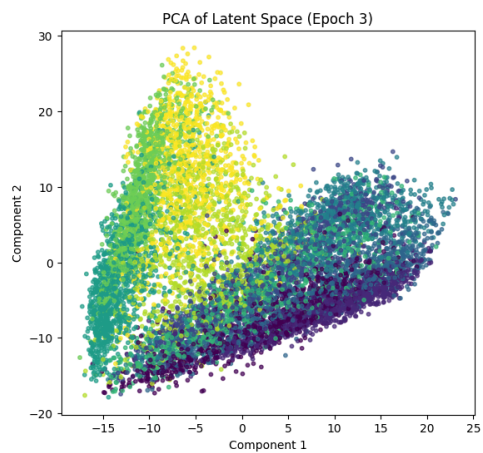
# Get the latent space representations
latent_space_data = encoder_model.predict(X_test_flat)

# PCA Visualization
visualize_latent_space(
    latent_space_data, f"PCA of Latent Space (Epoch {epoch + 1})", method='pca'
)

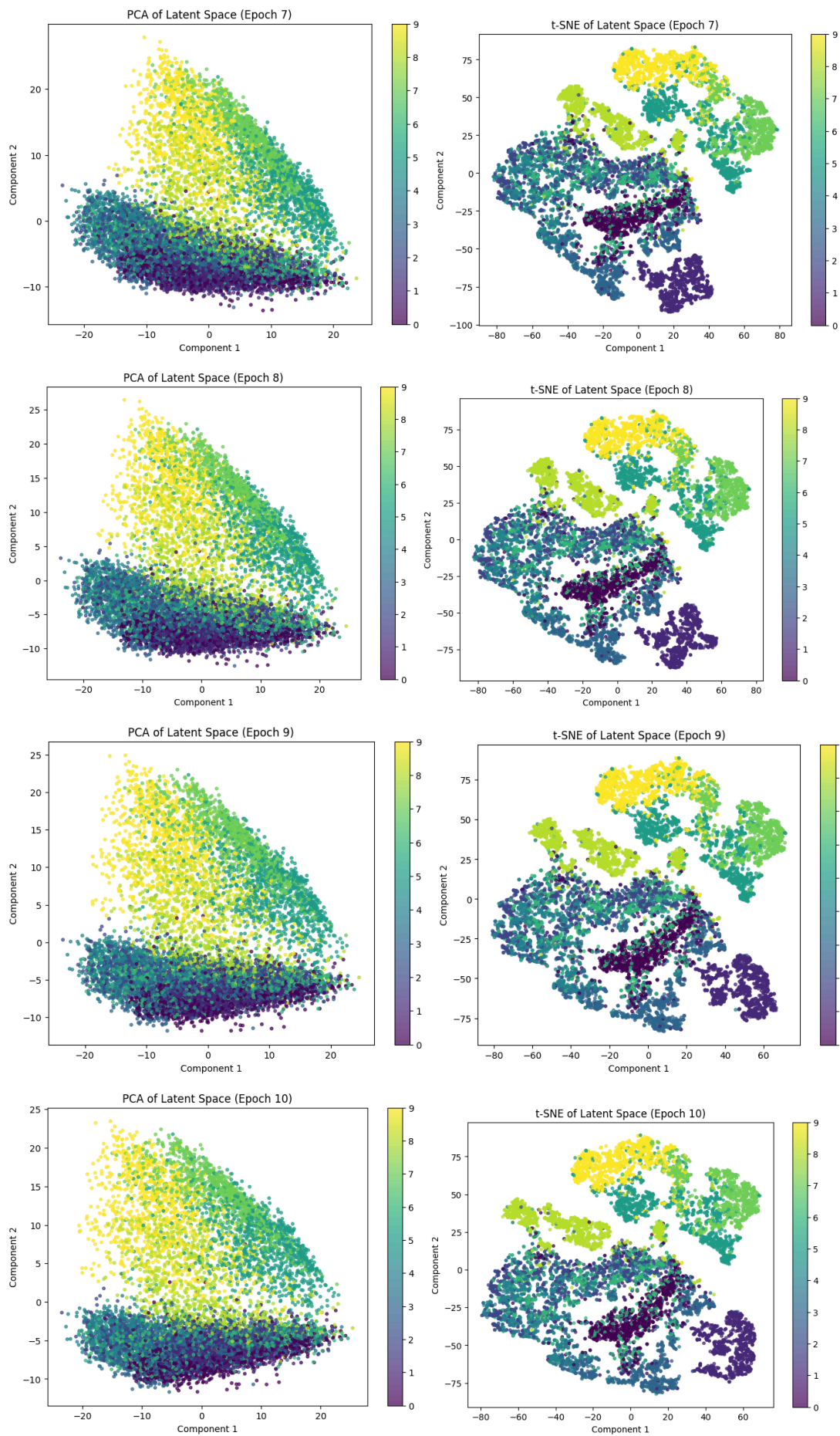
# t-SNE Visualization
visualize_latent_space(
    latent_space_data, f"t-SNE of Latent Space (Epoch {epoch + 1})", method='tsne'
)

```









**INFERENCE**